URBANFIX · BIG DATA ANALYTICS

Monsoon Intelligence

User guide & technical overview of BDA in UrbanFix

Kafka · HDFS data lake · Spark SQL · Spark MLlib · Structured Streaming

Generated 25 October 2026 from the live gold tables

In one line: UrbanFix fuses a full year of **real Mumbai rainfall** with civic complaints across **24 BMC wards** and finds that complaints surge **4.6x** about **48 hours** after heavy rain. The municipal dashboard turns that into actions: where to send crews, which wards need drainage investment, and how many duplicate dispatches to avoid.

4.6x @ 48h peak surge after heavy rain	**5,163** real incidents from 18,455 reports	**72%** crew dispatches saved	**6 of 24** critical-risk wards

Contents

1. What BDA does in UrbanFix
2. Data sources and the honesty rule
3. Architecture
4. Pipeline stages
5. Key results
6. Dashboard user guide (Monsoon Intelligence)
7. Use cases — how the municipality acts
8. How BDA connects to the UrbanFix app
9. Running and refreshing the pipeline
10. API reference
11. Limitations, glossary and viva talking points

1. What BDA does in UrbanFix

UrbanFix lets citizens report civic problems — potholes, garbage, waterlogging, broken streetlights — from a phone, with AI filling in the category, severity and department. That is the **operational** side. The BDA layer is the **analytical** side: it looks at all reports together, joins them with outside data, and answers questions no single report can.

The central question

Do civic complaints in a ward surge after heavy rain — and if so, how much, how soon, and where?

Mumbai's monsoon (June–September) brings flooding, potholes and blocked drains every year. If the municipality knows **when** the wave of complaints will arrive and **which wards** are hit hardest, it can position crews and materials before the backlog builds instead of reacting after it.

What BDA adds on top of the normal app

- **Fusion** — joins complaints with real daily rainfall per ward (two independent datasets) at ward × day grain.
- **Lag analysis** — measures how many hours after rain the complaint peak arrives (24h / 48h / 72h).
- **Civic Vulnerability Index (CVI)** — ranks the 24 wards by rain surge, recurrence and drainage deficit.
- **Deduplication at scale** — a spatio-temporal graph merges many reports of one problem into a single incident.
- **Forecasting** — a Spark MLlib model predicts ward incidents from rainfall; a live outlook applies it to the real forecast.
- **Data quality gate** — veracity checks reject bad GPS, out-of-city and out-of-window reports before analysis.

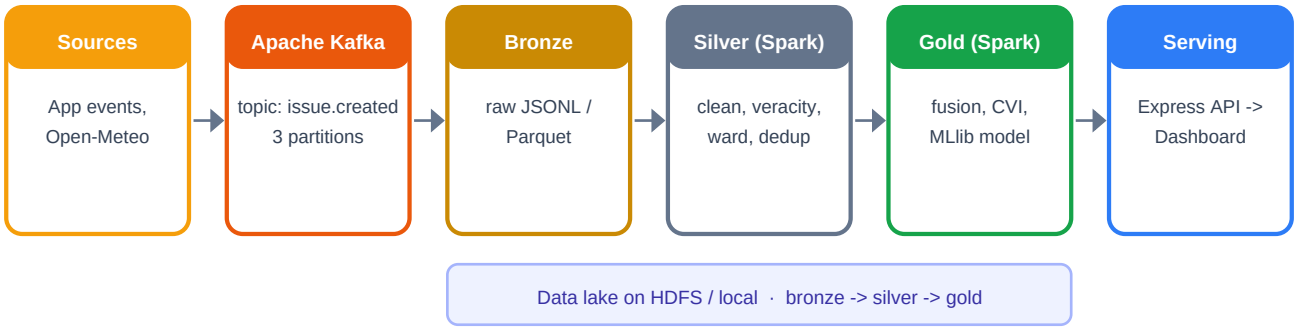
2. Data sources and the honesty rule

Dataset	Real or simulated	Source	Size
Daily rainfall	Real	Open-Meteo ERA5 reanalysis, 5 Mumbai stations (Colaba, Santacruz, Borivali, Chembur, Mulund)	1,830 station-days
Ward reference	Real	24 BMC administrative wards: centroid, zone, drainage capacity (mm/h), road km, Census 2011 population	24 wards
Civic reports	Simulated (is_simulated = true)	Generated in the UrbanFix event schema and streamed through Kafka	19,282 events
Road-damage images	Real metadata sample	Road damage image catalog used for the image-cleaning jobs	120 images
Live forecast	Real	Open-Meteo forecast API (3 past + 4 forecast days), fetched on demand	5 stations

The honesty rule. Rainfall and wards are real. Complaints are simulated — a city's real complaint log is not public — and every simulated row carries `is_simulated = true`. The generator couples complaints to the real rain series, but the Spark fusion job **never reads the generator's parameters**: it measures the relationship independently. Two reference files exist only for verification and are off-limits to the pipeline. So the 48-hour lag is *recovered* by Spark, not asserted. When the app's real reports accumulate, the same pipeline runs on them unchanged.

3. Architecture

Data flow: Kafka -> data lake (medallion) -> Spark -> API -> dashboard



The mobile app (citizens) and the municipal dashboard (admins) sit at both ends: reports flow in, decisions flow out.

Medallion data lake

Layer	What it holds	Format
Bronze	Data exactly as it landed: Kafka event batches, raw rainfall, raw image metadata. Never edited — it is the audit trail.	JSONL, Parquet
Silver	Cleaned and typed: timestamps parsed, GPS checked, each report attributed to a ward, duplicates linked into incidents.	Parquet (partitioned by year)
Gold	Business-ready marts: ward × date fusion, lag surge, vulnerability index, ML forecasts and audit reports.	Parquet + CSV/JSON

The lake runs on the local filesystem by default so the project works on a laptop; setting `URBANFIX_LAKE=hdfs` points every Spark read and write at the Hadoop cluster instead (same bronze/silver/gold layout under `/urbanfix/lake`). Spark runs inside WSL (Ubuntu, JDK 17) because the Windows JVM on this machine cannot open the loopback socket Spark needs.

Technology map

Big-data concept	Used for	Where
Apache Kafka (producer/consumer, partitions)	Streaming civic events into the lake	bda/kafka/
HDFS (namenode + datanode)	Distributed storage for the lake	bda/hdfs/, docker-compose.yml
Spark SQL + window functions	Cleaning, ward attribution, lag features, fusion join	spark/03, 05, 06
Graph-style dedup (connected components)	Merging duplicate reports into incidents	spark/05_events_silver.py
Spark Structured Streaming	Live windowed counts and ward alerts on the Kafka stream	spark/07_stream_pulse.py
Spark MLlib (RandomForest, LinearRegression)	Forecasting incidents from rainfall	spark/08_mllib_forecast.py
REST serving layer	Exposes gold tables to the dashboard	backend/routes/bdaRoutes.js

4. Pipeline stages

#	Stage	Tech	What it does	Writes
K	Kafka producer -> consumer	Kafka	Streams report events through the broker; consumer lands them in bronze	bronze/events/
R	Rainfall ingest	Python	Pulls one year of real daily rainfall for 5 stations	bronze/weather/
01	Ingest image metadata	Spark SQL	Reads the image catalog into a Parquet spine	bronze/image_metadata/
02	EDA	Spark SQL	Profiles labels, corruption, duplicates, class balance	gold/analytics/eda_summary
03	Clean images	Spark SQL	Removes corrupt, unlabelled and duplicate images	silver/cleaned_images/
04	Analytics scorecards	Spark SQL	Severity, status and ward hotspot scorecards	gold/analytics/*.csv
05	Events -> silver	Spark SQL	Veracity gate, ward attribution, duplicate graph	silver/events_clean/, veracity_report, dedup_audit
06	Rain × complaint fusion	Spark SQL, windows	Ward × date join, lag surge, Civic Vulnerability Index	gold/fusion/
07	Live city pulse	Structured Streaming	Windowed counts on the Kafka stream, alerts above a threshold	console + checkpoint
08	Complaint forecast	Spark MLlib	RandomForest + LinearRegression on lagged rain; scenario grid	gold/ml/

5. Key results

5.1 Complaints peak 48 hours after heavy rain

Spark compares the average complaints on days after heavy rain (> 45 mm) with each ward's dry-day baseline (< 5 mm), averaged over wards:

Lag after heavy rain	Surge vs dry days	Heavy-rain ward-days
24 hours	3.89x	376
48 hours	4.59x (peak)	376
72 hours	4.09x	376

Problems do not show up the moment it rains — water has to pool, drains back up, potholes open under traffic and people report over the following days. The practical meaning: **a crew dispatched the morning after heavy rain arrives ahead of the peak**.

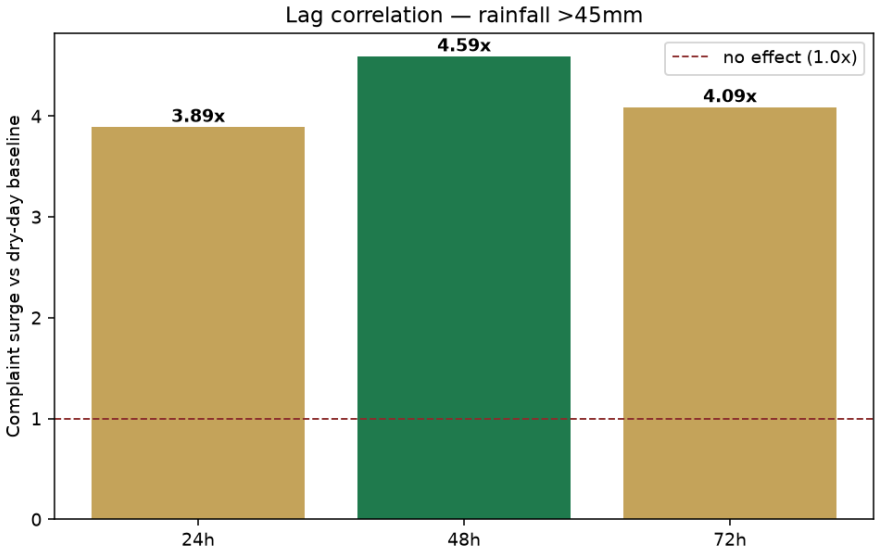


Figure 1 — Complaint surge at 24, 48 and 72 hours after heavy rain (Spark job 06).

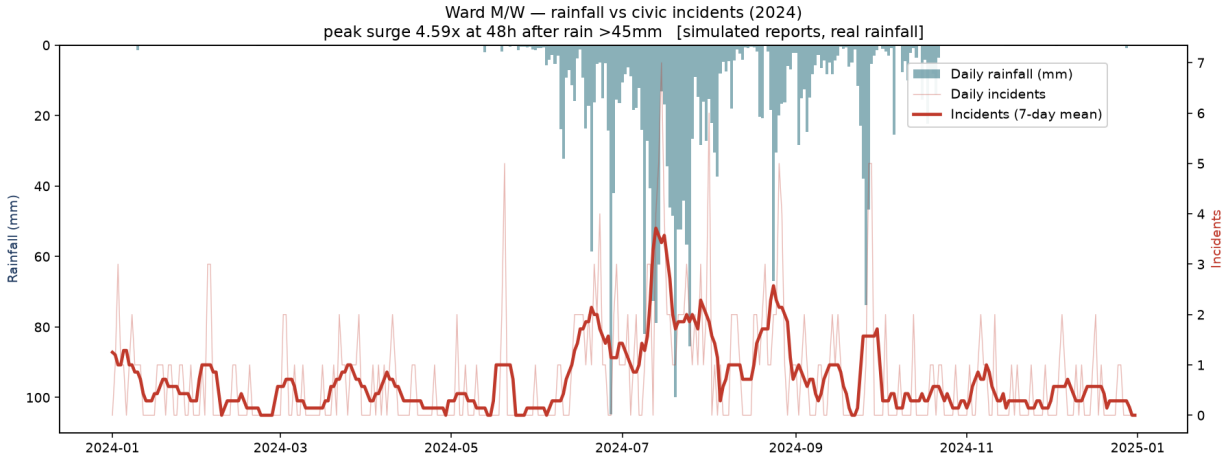


Figure 2 — Ward M/W (Chembur, the top-risk ward): daily rainfall against incidents for 2024. The complaint wave follows the monsoon block.

5.2 Data quality and deduplication

Stage	Reports
Landed in bronze (from Kafka)	19,282
Rejected: GPS accuracy too poor	-583
Rejected: outside Mumbai bounding box	-191
Rejected: outside analysis window	-62
Passed veracity (silver)	18,455
Quarantined: more than 5 km from any ward centroid	976
Distinct incidents after graph dedup	5,163 (3.57 reports per incident)

Deduplication builds a graph whose edges join reports of the same type that are close in space and time (136,257 edges), then takes connected components as incidents. Result: **72% fewer work orders** for the same set of real problems. Department labels were also normalised from 11 variants to 7.

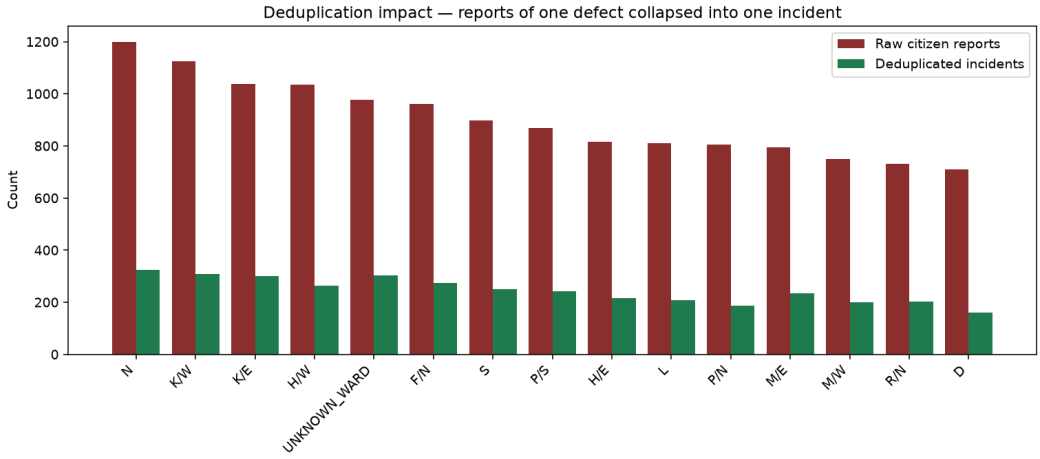


Figure 3 — Raw reports vs deduplicated incidents per ward.

5.3 Which wards are most vulnerable

The **Civic Vulnerability Index** combines three min-max-normalised terms: **50%** rain surge at 48h, **35%** recurrence (how often the ward surges) and **15%** drainage deficit (low drainage capacity). The top quartile is CRITICAL.

Rank	Ward	Zone	48h surge	Recurrence	Drainage (mm/h)	CVI	Tier
1	M/W — Chembur	Eastern Suburbs	6.27x	15	35	0.835	CRITICAL
2	D — Grant Road / Malabar Hill	Island City	5.85x	15	25	0.831	CRITICAL
3	L — Kurla	Eastern Suburbs	5.18x	18	30	0.803	CRITICAL
4	B — Sandhurst Road / Dongri	Island City	6.23x	12	25	0.784	CRITICAL
5	F/N — Matunga / Sion	Island City	5.61x	15	30	0.765	CRITICAL
6	H/E — Bandra East / Santacruz East	Western Suburbs	5.33x	17	35	0.761	CRITICAL
7	G/N — Dadar / Mahim	Island City	4.99x	13	30	0.599	MODERATE
8	N — Ghatkopar	Eastern Suburbs	4.84x	14	35	0.581	MODERATE

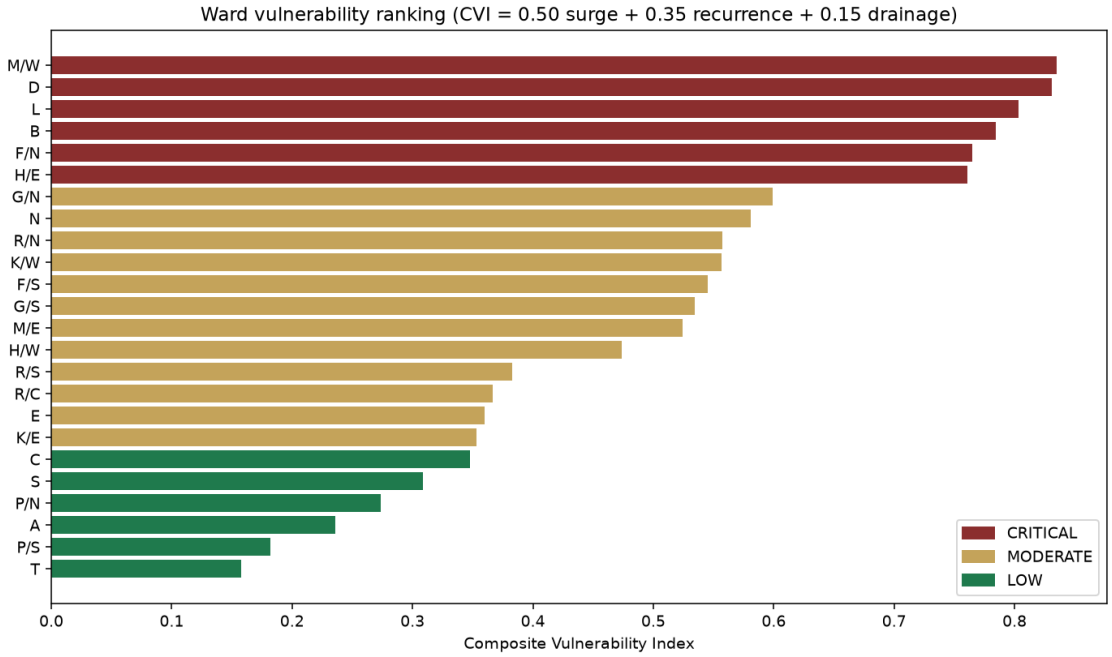


Figure 4 — Civic Vulnerability Index by ward.

5.4 Forecast model (Spark MLlib)

Question: *How many incidents will a ward log, given rain today and the previous 3 days?* Trained on 7,104 ward-days and tested on 1,680 held-out ward-days (blocked hold-out: every 5th ISO week is test — whole weeks are held out so one storm never sits on both sides).

Model	RMSE	MAE	R^2
Random Forest	1.016	0.770	0.263
Linear Regression (best)	1.006	0.757	0.278
Baseline (ward average)	1.172	0.844	0.020

Both models beat the 'same as usual' baseline: **14% lower error** overall and **23% lower on rainy days**, when planning matters. R^2 stays modest because daily counts per ward are small and noisy.

Feature	Importance
ward	29.8%
rain_d2	22.5%
rain_d3	15.3%
rain_d1	10.0%
rain_2d	7.4%
rain_d0	6.6%
drainage_capacity	5.0%
month	2.8%
is_monsoon	0.5%

Rain from two days earlier (rain_d2) is the strongest rain feature — the model rediscovers the 48-hour lag on its own, independently of the fusion job.

6. Dashboard user guide — Monsoon Intelligence

Opening it

- Start the backend (`backend/: node server.js`, port 5000) and the dashboard (`municipal-admin/: npm run dev`, port 3000).
- Open `http://localhost:3000` and sign in with the municipal admin account.
- Click **Monsoon Intelligence** in the left sidebar. Nine tabs run across the top.

Tab	What it shows	How to use it
Overview	The headline finding with lag bars, four KPIs, the live pipeline diagram (with real Kafka partition counts and lake sizes), use-case cards and the critical wards.	Start here in a meeting: one screen tells the story.
Live Outlook	Today plus the next 3 days from the real Open-Meteo forecast for 5 stations; expected incidents per ward from the MLlib model; a status banner (Normal / Elevated / Surge expected), a ward heat map and the rain-gauge grid.	Check every morning in monsoon season. Click a day to see that day's ward outlook.
Rain × Complaints	Daily rainfall bars against the incident line for the whole city or one ward (dropdown), with a zoom handle; a rain-band response chart at +0/24/48/72h; the Spark lag analysis; seasonality; the worst heavy-rain episodes; the 5 rain gauges.	Pick a ward to see its own history. Drag the brush to zoom into July.
Ward Risk	Map of the 24 wards (switch between CVI, incident volume and 48h surge), a detail panel for the selected ward, and the full ranked table with the three CVI components as bars.	Click a ward on the map or in the table; use 'See this ward's history' to jump to its fusion chart.
Surge Planner	Enter a rainfall forecast (slider or presets) and a horizon. It returns expected incidents per ward from historical ward-days in the same rain band, a heat map, and a dispatch list that flags the top quarter for pre-positioning.	Use the evening before forecast heavy rain to decide where crews start.
MLlib Forecast	Model comparison table, feature importance, actual vs predicted on held-out weeks, scenario curves and the wards most sensitive to 65 mm of rain.	Use to explain and trust the forecast, and in the viva.
Patterns	Weekday × hour heat map of when citizens report, department × month load, category severity mix, GPS accuracy and lifecycle status.	Staff the triage desk to match peak hours; plan department capacity by month.
Data Quality	Veracity funnel, dedup by ward, Kafka partition balance, lake layer sizes and the image-cleaning result.	Prove the numbers are trustworthy before acting on them.
Pipeline & Reports	Every stage with its status and when it last wrote output, plus a gallery of the charts the Spark jobs generated.	Check freshness after re-running the pipeline; open figures for reports.

7. Use cases — how the municipality acts on it

Monsoon morning routine (control room)

1. Open **Live Outlook**. If the banner says Elevated or Surge expected, note the peak day.
2. Read the ward list for that day — wards at 1.5x or more of their normal load are on alert.
3. Open **Surge Planner**, set the forecast rain and +48h, and assign the 'Pre-position crew' wards first.
4. In the main dashboard, assign field workers with deadlines; workers see the tasks in the mobile app.

Drainage investment priority (planning cell)

1. Open **Ward Risk**, mode Vulnerability (CVI).
2. Shortlist CRITICAL wards; check whether their score comes from surge, recurrence or drainage deficit.

3. A high drainage-deficit ward is a capital-works candidate; a high-surge, adequate-drainage ward points to maintenance (desilting).

Avoiding duplicate dispatches (operations)

1. **Data Quality** shows 3.6 reports per real incident on average.
2. The mobile app's duplicate check uses the same idea live: a citizen near an existing report is offered 'Add my report to it'.
3. Work orders are raised per incident, not per report — the same crew-hours fix more problems.

Staffing and capacity (administration)

1. Open **Patterns**: the weekday × hour heat map shows when reports arrive; staff triage for those hours.
2. The department × month table shows which departments need extra capacity in which months.

8. How BDA connects to the UrbanFix app

- **Citizen mobile app** — a report (photo, GPS, category) is analysed by Gemini on Vertex AI and stored in Supabase. The same event shape (`issue.created`: category, severity, department, GPS + accuracy, timestamps) is what the Kafka producer streams, so real app reports can flow into the lake without changes.
- **AI vs BDA** — Vertex AI handles *one report at a time* (what is in this photo?). BDA handles *all reports together* (when and where do problems cluster?). They are complementary.
- **Municipal dashboard** — the operational pages (issues, map, workers) run on live Supabase data; Monsoon Intelligence runs on the BDA gold tables through `/api/bda/*`.
- **Field workers** — admins assign tasks from the dashboard (worker + deadline); workers start work and resolve with an after-photo in the app; the citizen is notified at each step.

9. Running and refreshing the pipeline

End to end, from Windows (Spark runs in WSL):

```
MSYS_NO_PATHCONV=1 wsl -d Ubuntu -- bash
/mnt/c/Work/Projects/FullStackAI/UrbanFix/bda/run_bda.sh all
```

Individual stages:

```
run_bda.sh reference | generate | images | fusion | reports | kafka | stream | hdfs
```

Forecast model (job 08):

```
MSYS_NO_PATHCONV=1 wsl -d Ubuntu -- bash
/mnt/c/Work/Projects/FullStackAI/UrbanFix/bda/scripts/run_ml.sh
```

The dashboard API caches the gold tables and rebuilds automatically when `ward_date_fused_gold.csv` changes. The live outlook refreshes its forecast every 30 minutes. Run order matters: reference -> generate -> fusion (05, 06) -> 08.

10. API reference

All endpoints need an admin JWT (`Authorization: Bearer ...`) except figures.

Endpoint	Returns
GET /api/bda/overview	Headline findings, lag, layer sizes, Kafka profile, veracity, provenance
GET /api/bda/fusion?ward=M/W	Daily rain × incidents (city or ward), monthly, rain-band response by lag, episodes, stations
GET /api/bda/wards	Ward mart: geography, CVI and components, tier, yearly totals

Endpoint	Returns
GET /api/bda/planner?rain=65&lag=2	Expected incidents per ward for a rain amount and horizon, with the method
GET /api/bda/ml	MLlib metrics, importances, hold-out predictions, scenario curves
GET /api/bda/outlook	Live Open-Meteo forecast and expected incidents per ward for the next days
GET /api/bda/patterns	Hour × weekday heat map, department × month, categories, accuracy, status
GET /api/bda/quality	Veracity, dedup audit, image cleaning, ingest and layer stats
GET /api/bda/jobs	Pipeline stage catalog with output freshness, list of figures
GET /api/bda/figures/:name.png	A chart generated by the Spark jobs

11. Limitations, glossary and viva talking points

Known limitations

- Civic reports are simulated (flagged); rainfall and wards are real. The pipeline is ready for real app data.
- Ward attribution is nearest-centroid with a 5 km cutoff, not polygon boundaries; reports beyond it are quarantined.
- Rainfall is daily, so the lag resolution is 24 hours.
- The forecast predicts noisy daily counts: it beats the baseline clearly but R^2 is modest — use it for ranking and planning, not exact numbers.
- The live outlook superposes the model's single-storm responses over the last four days — an approximation of multi-day storms.

Glossary

Term	Meaning
Bronze / Silver / Gold	Raw -> cleaned -> business-ready layers of the data lake (medallion architecture)
Fusion	Joining two independent datasets (complaints and rainfall) at a common grain: ward × day
Surge ratio	Average complaints after heavy rain ÷ average on dry days, for the same ward
Lag	Hours between the rain and the complaint response it causes
CVI	Civic Vulnerability Index: 0.50 surge + 0.35 recurrence + 0.15 drainage deficit (normalised)
Veracity	Checks that a report is trustworthy enough to analyse (GPS accuracy, inside the city, inside the window)
Incident	One real problem; many reports of it are merged by dedup
RMSE / MAE / R^2	Forecast error (lower is better) and explained variance (higher is better)

Viva talking points

- **Volume, velocity, variety, veracity, value**: 19,282 events; Kafka streaming + Structured Streaming; events, weather and images; a veracity gate that rejected 827 reports; decisions on crews and drainage.
- The lag is **recovered, not assumed**: the fusion job never reads the generator's parameters, and the MLlib model independently ranks 2-day-old rain highest.
- Dedup turns 18,455 reports into 5,163 incidents — 72% fewer dispatches.
- Blocked hold-out (every 5th week) avoids leaking a storm's neighbouring days between train and test.
- Same architecture scales from a laptop (local lake) to a cluster (HDFS) by one environment variable.